American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Intermediate Division

PROBLEM: Given a string to create 2 parallel arrays, process the letters in the string from left to right, inserting each letter into the first array in alphabetical order, one letter at a time. During this process, each letter is assigned an integer value in the second array as follows:

1. The first letter has a value of 0.
2. If the new letter is the first or last in the array, its value is one more than the value of the adjacent letter.
3. Otherwise, it is one more than the larger value of the two adjacent values. If the letter is already in the array, the new letter is placed before the existing letter.

Once the two arrays are created, print two different strings separated by a single space. For each letter in the array from left to right:

- Check if there is a letter to the left of it that has a value that is one greater than its value. Stop if you encounter any value that is less than its value. If a letter meets the condition above, add it to the first string.
- Check if there is a letter to the right of it that has a value that is one greater than its value. Stop if you encounter any value that is less than its value. If a letter meets the condition above, add it to the second string.
- Do not add the letter if it would be in both strings.

If either string is empty, print NONE instead.

EXAMPLE #1:

Here is how the array is built for the string **PYTHONN**:

The letter P illustrates rule 1; Y and H illustrate rule 2; the T, O and Ns illustrate rule 3; and the final N illustrates what happens if there is a duplicate letter.

P	*Letters*	P								
	Values	0								
Y	*Letters*	P	Y							
	Values	0	1							
T	*Letters*	P	T	Y						
	Values	0	2	1						
H	*Letters*	H	P	T	Y					
	Values	1	0	2	1					

O	*Letters*	H	O	P	T	Y				
	Values	1	2	0	2	1				

N	*Letters*	H	N	O	P	T	Y			
	Values	1	3	2	0	2	1			

N	*Letters*	H	N	N	O	P	T	Y		
	Values	1	4	3	2	0	2	1		

In these final arrays, process each letter as follows:

- The letter H has a value of 1. Searching to its left, no letter is found. Searching to its right, a value of 2 is found. Add H to the second string.

- The 1st N has a value of 4. Searching to its left, there is no 5. Searching to its right, no 5 is found before encountering the 3. It is not added to either string.

- The 2nd N has a value of 3. Searching to its left, there is a 4. Searching to its right, there is no 4 before encountering the 2. Add N to the first string.

- The O has a value of 2. Searching to its left, there is a 3. Searching to its right, there is no 3 before encountering the 0. Add O to the first string.

- The P has a value of 0. Searching to its left, there is a 1. Searching to its right, there is a 1. Do not add it to either string.

- The T has a value of 2. Searching to its left, there is no 3 before encountering the 0. Searching to its right, there is no 3. It is not added to either string.

- The Y has a value of 1. Searching to its left, there is a 2. Searching to its right, no letter is found. It is added to the first string.

The output would therefore be NOY H.

EXAMPLE #2: The string **BINARYSEARCHTREE** results in the following array:

Letters	A	A	B	C	E	E	E	H	I	N	R	R	R	S	T	Y
Values	2	1	0	3	5	4	2	3	1	2	5	4	3	5	6	4

Following the same process as above, the output is AERY CNS.

INPUT: Input a string to use in building the two arrays. The string will be no more than 80 characters containing all capital letters.

OUTPUT: Output the two strings separated by a single space.

SAMPLE INPUT:

1. PYTHONN
2. BINARYSEARCHTREE
3. CORONAVIRUS
4. FINALSFORACSL
5. HACKERRANKPLATFORM

SAMPLE OUTPUT:

1. NOY H
2. AERY CNS
3. NOUV NONE
4. FLS IOR
5. AR CEL

American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Intermediate Division

TEST DATA

TEST INPUT:

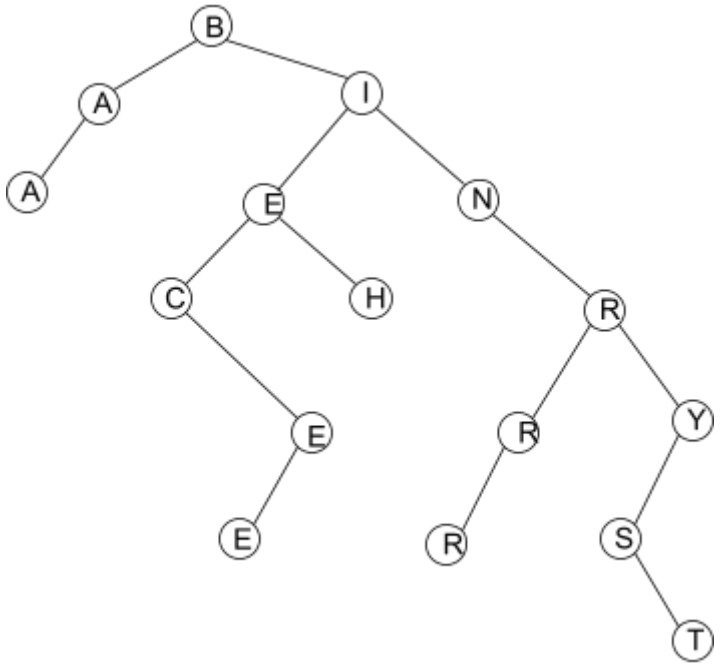
1. MOTHER
2. ACSLCONTEST
3. SUPERCALIFRAGILISTIC
4. JAVAPROGRAMMING
5. ABDFHKMOQTVWYZ

TEST OUTPUT:

1. HT O
2. T A
3. CIILU FG
4. AORV NONE
5. NONE ABDFHKMOQTVWY

FOR INFORMATION PURPOSES ONLY

The procedure above is a way to create a binary search tree. The values in the array represent the depth of the nodes. The tree below is the result:



The letters printed are those with only a left child followed by those with only a right child. The relative value of each letter determines if a node is a left child or a right child. In these final arrays, the 2nd A and 2nd E are in the first string because they have only a left child while the C, N, and S are in the second string because they have only a right child.

American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Intermediate Division

问题: 给定一个字符串，用于创建 2 个平行数组，从左到右处理字符串中的字母，按字母表顺序将字母依次插入第一个数组，每次一个字母。在此过程中，在第二个数组中为每个字母分配的一个整数值，规则如下所示：

1. 第一个字母的数值是 0。
2. 如果新插入的字母是数组中的第一个或最后一个，则它的数值比相邻字母的数值大 1。
3. 否则，它比两个相邻值中的较大值大 1。如果这个字母已经在数组中，则将新插入的字母放在现有字母之前。

创建两个数组后，打印输出两个不同的字符串，用一个空格隔开。对于数组中从左到右的每个字母：

- 查看它的左侧，是否有一个字母的值比它大 1 。如果遇到任何小于它的值，请停止。如果一个字母满足上述条件，请将它添加到第一个字符串中。

- 查看它的右侧，是否有一个字母的值比它大 1 。如果遇到任何小于它的值，请停止。如果一个字母满足上述条件，请将它添加到第二个字符串中。

- 如果一个字母同时符合两个字符串的条件，则不要添加这个字母。

如果任一字符串为空，则打印输出 NONE。

示例 #1:

下面展示了如何对字符串 **PYTHONN** 创建数组：

字母 P 表示规则 1； Y 和 H 表示规则 2；T、O 和 N 表示规则 3；最后一个 N 表示如果有重复的字母会如何在数组中处理。

P	*字母*	P								
	数值	0								
Y	*字母*	P	Y							
	数值	0	1							
T	*字母*	P	T	Y						
	数值	0	2	1						
H	*字母*	H	P	T	Y					
	数值	1	0	2	1					

O	*字母*	H	O	P	T	Y				
	数值	1	2	0	2	1				

N	*字母*	H	N	O	P	T	Y			
	数值	1	3	2	0	2	1			

N	*字母*	H	N	N	O	P	T	Y		
	数值	1	4	3	2	0	2	1		

在最终的数组中，按如下方式处理每个字母：

- 字母 H 的数值为 1。搜索它的左侧，没有任何字母。搜索它的右侧，找到数值 2。将 H 添加到第二个字符串。
- 第一个 N 的数值为 4。搜索它的左侧，没有数值 5。搜索它的右侧，搜索到 3 之前搜索不到 5。N 不会添加到任何字符串中
- 第二个 N 的数值为 3。搜索它的左侧，有一个数值 4。搜索它的右侧，在搜索到 2 之前搜索不到 4。将 N 添加到第一个字符串。
- O 的数值为 2。搜索它的左侧，有一个数值 3。搜索它的右侧，在搜索到 0 之前搜索不到 3。将 O 添加到第一个字符串。
- P 的数值为 0。搜索它的左侧，有一个数值 1。搜索它的右侧，也有一个数值 1。不能将它添加到任何字符串中
- T 的数值为 2。搜索它的左侧，在搜索到 0 之前搜索不到 3。搜索它的右侧，没有搜索到 3。它不会添加到任何字符串中。
- Y 的数值为 1。搜索它的左侧，有一个数值 2。搜索它的右侧，没有找到任何字母。将它添加到第一个字符串中。

最终输出将会是 NOY H。

示例 #2: 字符串 **BINARYSEARCHTREE** 会得到下列数组：

字母	A	A	B	C	E	E	E	H	I	N	R	R	R	S	T	Y
数值	2	1	0	3	5	4	2	3	1	2	5	4	3	5	6	4

按照以上相同的过程，输出将会是 AERY CNS。

输入: 输入一个字符串用于创建 2 个数组，这个字符串中的所有字母均为大写字母，且不超过 80 个字母。

输出: 输出由一个空格隔开的两个字符串。

样本输入：

1. PYTHONN
2. BINARYSEARCHTREE
3. CORONAVIRUS
4. FINALSFORACSL
5. HACKERRANKPLATFORM

样本输出：

1. NOY H
2. AERY CNS
3. NOUV NONE
4. FLS IOR
5. AR CEL

American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Intermediate Division

测试数据

测试输入：

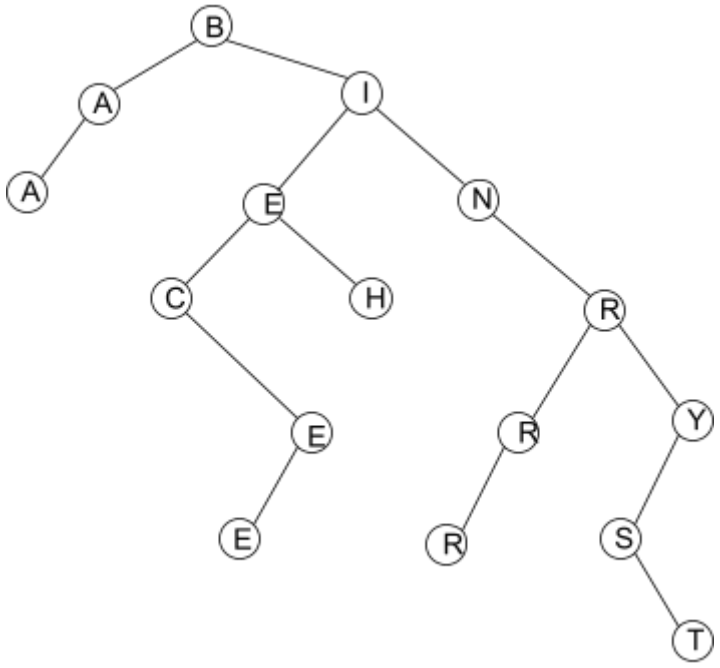
1. MOTHER
2. ACSLCONTEST
3. SUPERCALIFRAGILISTIC
4. JAVAPROGRAMMING
5. ABDFHKMOQTVWYZ

测试输出：

1. HT O
2. T A
3. CIILU FG
4. AORV NONE
5. NONE ABDFHKMOQTVWY

FOR INFORMATION PURPOSES ONLY

The procedure above is a way to create a binary search tree. The values in the array represent the depth of the nodes. The tree below is the result:



The letters printed are those with only a left child followed by those with only a right child. The relative value of each letter determines if a node is a left child or a right child. In these final arrays, the 2nd A and 2nd E are in the first string because they have only a left child while the C, N, and S are in the second string because they have only a right child.